Runtime-Side Security Profile Generation for OCI Containers

Ivan Platonov[1], Andrey Sadovykh[2], and Kirill Saltanov[3]

[1] Innopolis University, Innopolis, Russia
i.platonov@innopolis.university
[2] Softeam, Paris, France
andrey.sadovykh@softeam.fr
[3] Innopolis University, Innopolis, Russia
kirill.saltanov@innopolis.university

Abstract. With still growing popularity of OCI containers, and raising popularity of AI offensive technologies, the need for protection of containerized workloads from malicious use is becoming more and more imporatant. Currently, there are 3 approaches to secure containers from malicious use: sandboxing, manual policy authoring, and cluster-side recorders. And all of them have their own trade-offs. Sandboxed runtimes such as gVisor reduce kernel exposure by interposing a user-space kernel or a microVM, yet impose substantial CPU, memory, and I/O overhead. Manual policy authoring requires kernel-level expertise and per-workload validation effort that most engineering teams cannot afford to sustain. Cluster-side recorders such as the Kubernetes Security Profiles Operator and Inspektor Gadget require Kubernetes cluster + recorder + storage; DevSecOps engineer or experienced QA team; profile-pinning workflow. This paper presents a security-oriented fork of `runc` in which profile generation is performed by the runtime itself. On a designated scanning invocation, the runtime executes the workload under in-memory relaxation of its confinement, traces the resulting kernel interactions through eBPF-based syscall and cgroup-scoped capability recorders combined with AppArmor learning mode, and writes a workload-specific seccomp allow-list, AppArmor profile, and minimised capability set back into the OCI bundle. Subsequent runs enforce these artefacts as ordinary OCI inputs, requiring no recorder stack, daemon, or cluster-side tooling; tracing logic ships as hidden sub-commands of the runtime binary, so the bundle gains data artefacts only and remains portable. The runtime is evaluated against unmodified `runc` and gVisor along three axes: runtime overhead, profile quality, and resistance to representative container-escape classes. The result [placeholder for now]

Keywords: OCI runtime · runc · seccomp · AppArmor · Linux capabilities · eBPF.

1 Introduction

Containers are now a standard deployment unit for cloud applications, but their isolation boundary remains fundamentally different from a virtual machine

boundary. A container usually shares the host kernel with all other containers on the node, so kernel-facing attack surface matters directly to tenant isolation and host integrity [12, 4]. The Open Container Initiative (OCI) standardises the runtime contract, and `runc` remains the reference implementation used under many higher-level container stacks. Its strength is compatibility and low overhead; its weakness is that strong confinement depends on policies supplied by the operator.

Linux already exposes the mechanisms required for a narrower boundary. Seccomp filters syscalls, AppArmor restricts path-based filesystem access, and Linux capabilities split privileged root operations into smaller units. The problem is not the absence of mechanisms, but the cost of producing profiles that are strict enough to be useful and still broad enough not to break the workload. In practice, profile authoring often requires kernel knowledge, representative tests, and repeated debugging. Small teams therefore tend to keep default profiles or disable hardening when it conflicts with delivery pressure.

This paper asks whether the runtime itself can reduce that cost. The proposed system is a fork of `runc` with a scanning mode invoked as part of a normal runtime command. During a scan, the runtime relaxes the bundle specification in memory, installs recording hooks, executes the workload, and writes generated artefacts back to the bundle. The scan is not an enforcement run; it is a learning phase intended for local testing or continuous integration. Subsequent runs use the generated seccomp, AppArmor, and capability policies as normal runtime inputs.

The contribution is threefold. First, the runtime exposes profile generation as a single-flag workflow rather than a separate recorder stack. Second, it combines three confinement mechanisms in one pipeline: syscall recording through `oci-seccomp-bpf-hook`, cgroup-wide capability tracing through BCC and `bpftool`, and AppArmor learning in complain mode. Third, it keeps scanner hooks inside the runtime binary itself, preserving the OCI bundle's portability and avoiding generated scripts in the bundle directory.

The remainder of the paper is organised as follows. Section 2 describes the runtime design and implementation. Section 3 defines the evaluation methodology. Section 4 analyses the expected security and operational trade-offs. Section 5 positions the work against sandboxed runtimes, manual hardening, and cluster-side recorders. Section 6 concludes.

2 Implementation

2.1 Runtime Architecture

The implementation extends the normal `runc run` and `runc create` paths with an optional scanning branch. If the user does not pass `--security-scan`, the runtime executes the bundle using the ordinary OCI specification. If the flag is present, the runtime performs five steps: it relaxes confinement in memory, installs hooks, runs the container, stops tracers, and finalises the generated profile set.

The relaxation step is deliberately in-memory. The on-disk `config.json` is not widened before the run. Instead, the runtime clears seccomp, clears the SELinux label, disables `NoNewPrivileges`, grants all known capabilities to the process, and replaces any AppArmor profile with a generated complain-mode profile. This makes the trace complete: a policy that remains active during recording would hide legitimate behaviour and produce a profile that breaks application.

The generated artefacts are written under `generated/` in the bundle directory. Table 1 summarises the stable output contract.

Table 1. Artefacts produced by a scan run.

Artefact	Purpose
`seccomp.json`	OCI seccomp allow-list produced by syscall tracing.
`capable-bpfcc.log`	Raw cgroup-wide capability trace consumed by finalisation.
`apparmor.profile`	Generated AppArmor profile: loaded in complain mode during the scan, then updated from collected audit denials for enforcement.
`apparmor-load.log`	Loader diagnostics from `apparmor_parser`.
`capabilities-from-proc-status.txt`	Diagnostic snapshot of granted capabilities.
`spec.original.json`	Backup of the pre-scan specification.

2.2 Capability Narrowing

The capability pipeline is the most important part of finalisation because it mutates the active `config.json`. The scanner runs BCC's `capable-bpfcc`, which observes `cap_capable` checks in the kernel [6]. A plain PID filter is insufficient because many workloads fork child processes after the container init has started. The implementation therefore uses `capable-bpfcc --cgroupmap`: it resolves the container cgroup, creates and pins a BPF map through `bpftool`, inserts the cgroup inode into that map, and starts the tracer against the pinned map.

Finalisation parses the capability log, normalises names to OCI `CAP_*` spelling, and replaces all five capability buckets: bounding, effective, permitted, inheritable, and ambient. The update is a rebuild from evidence, not a merge with previous or default capability sets: the final set starts empty and is extended only with capabilities observed by the scanner. If the trace contains no capability use, the resulting capability set is empty. This invariant prevents repeated scans from drifting toward broader defaults and matches the principle that a scan should not keep a privilege only because it appeared in an earlier configuration.

2.3 Seccomp and AppArmor Recording

Syscall recording is delegated to `oci-seccomp-bpf-hook`, a maintained OCI hook also used in container tooling for seccomp tracing [3]. The runtime sets the required tracing annotation and installs the hook as a prestart hook. The resulting `seccomp.json` is left as an explicit artefact for later enforcement rather than merged with any previous profile.

AppArmor recording follows a complain-mode workflow. The runtime writes a uniquely named profile, loads it with `apparmor_parser`, assigns it to the container process, and unloads it after the container stops. During post-stop finalisation it collects host audit denials for that profile, translates supported file and capability events into profile rules, and switches the generated profile to enforcement when rules were collected. If AppArmor is unavailable, the profile file is still generated, but the runtime reports that the host cannot load it.

2.4 Self-Exec Hooks

The current implementation registers hidden runtime sub-commands such as `scan-aa-load`, `scan-cap-trace-start`, and `scan-cap-trace-stop`. OCI hooks point back to the running `runc` binary, and parameters are passed through hook environment variables. Each sub-command drains hook state from standard input and writes diagnostics under `generated/`. The bundle therefore gains data artefacts only, not executable helper scripts.

3 Evaluation

The evaluation is designed to measure whether runtime-side profile generation can improve confinement while preserving the low overhead of a traditional OCI runtime. The comparison uses four configurations:

- default `runc`, representing the compatibility baseline;
- the proposed runtime with scan-generated enforcement profiles;
- gVisor representing user-space-kernel and microVM sandboxing.

The workload set combines small services and application-like targets. Minimal workloads such as `busybox`, `nginx`, and `redis` provide low-noise performance baselines. A larger application target such as XXXX provides a more realistic process tree and filesystem footprint. The matrix is intended to run on a fixedx Xxx, cgroup v2, BCC tools, `bpftool`, AppArmor utilities, and `oci-seccomp-bpf-hook`.

The measured dimensions are listed in Table 2. Performance numbers should be reported as medians and interquartile ranges after a warm-up run. Security-effectiveness measurements should be reported per attack class rather than collapsed into a single score, because different configurations block attacks at different layers.

Table 2. Evaluation metrics.

Metric family	Measurements
Performance	Startup latency, steady-state CPU, memory footprint, I/O bandwidth, and TCP throughput.
Profile quality	Number of allowed syscalls, observed capabilities, AppArmor log entries, and generated artefact size.
Security effectiveness	Blocked attempts for syscall-bound escapes, capability-bound escapes, filesystem breakouts, and runtime-CVE-inspired attacks.
Operational cost	Required host dependencies, number of commands, generated files, and compatibility failures.

At the time of writing, the implementation and reporting schema are complete, while the final benchmark matrix is still pending. For that reason, this paper treats quantitative tables as the evaluation contract rather than as completed empirical claims. The important methodological point is that the proposed runtime should be compared both with default `runc` and with stronger isolation systems, because it targets a middle point: lower operational cost than cluster-side recording and lower runtime overhead than full sandboxing.

4 Analysis

4.1 Security Properties

The scanner improves security by reducing the default privileges available to a later enforcement run. Seccomp removes unused syscalls from the kernel boundary; capability narrowing removes unused privileged operations; AppArmor adds path-based visibility and eventual confinement. These mechanisms are complementary: a capability profile can block privileged kernel checks, seccomp can remove entire syscall families, and AppArmor can restrict filesystem effects even when a syscall itself remains allowed.

The strongest invariant is that finalisation does not merge capability evidence with earlier broad defaults. A profile generator that unions current observations with previous capability sets would be operationally convenient but insecure by construction. The proposed runtime instead treats the scan as evidence: the output capability set is built from an empty set and only observed capabilities survive. This makes the scan sensitive to test coverage, but it also makes the result auditable.

The cgroup-wide tracer closes a common blind spot in dynamic profiling. PID-based recording can miss helpers, workers, shell pipelines, and language runtime children. Filtering by cgroup better matches the container abstraction and is especially important for workloads that spawn subprocesses after initialisation.

4.2 Limitations

The design inherits the limitations of dynamic tracing. A scan only records behaviour that actually happens during the scan window. Rare error paths, scheduled jobs, migration scripts, certificate renewal, and feature-flagged code can be missed. The practical mitigation is to run scans under end-to-end tests that exercise representative behaviour.

Root workloads are another limitation. Some capability checks are bypassed or under-reported for tasks running as UID 0, so scans of rootful workloads can produce incomplete capability evidence. The runtime warns about this case but does not rewrite the configured user, because doing so could change workload semantics.

Finally, AppArmor generation is automated for the common audit-denial cases but still requires validation for production use. The runtime collects denials, inserts supported file and capability rules, and can switch the generated profile from complain mode to enforcement. Manual review remains necessary for unhandled AppArmor operations, missed execution paths, and deciding whether the learned filesystem access is intentionally required or merely an artefact of the test run.

4.3 Operational Trade-Off

The proposed runtime does not replace sandboxed runtimes for high-assurance multi-tenant isolation. gVisor reduce host-kernel exposure more fundamentally by interposing a user-space kernel or a microVM [13, 10]. The runtime-side scanner instead targets the large class of deployments where default `runc` is attractive for performance and compatibility, but profile authoring is too expensive. Its value is reducing the setup cost from a recorder stack to a runtime flag.

5 Related Work

Prior work on container runtime comparison consistently shows that `runc` is fast but weaker as an isolation boundary, while gVisor trade overhead for stronger isolation [13, 11, 10]. Those studies usually compare against default `runc`. This paper instead asks what a traditional runtime can achieve when workload-specific policies are generated automatically.

Automated seccomp generation has been studied directly by Lopes et al. [8], and static or hybrid approaches can recover paths that pure tracing may miss [1]. The proposed runtime follows the dynamic tracing line but integrates it into the OCI runtime path and combines it with capabilities and AppArmor. Mattetti and Allouche [9] similarly argue for layered hardening across seccomp and mandatory access control.

Cluster-side recorders such as the Kubernetes Security Profiles Operator and tools such as Inspektor Gadget generate or assist with security profiles in Kubernetes environments [7, 5]. Tetragon uses eBPF for runtime observability and

enforcement-oriented monitoring [2]. These systems are powerful, but they assume cluster infrastructure and operational expertise. The proposed runtime addresses the same policy-generation problem one layer lower, for developers who can run a container but cannot operate a recorder stack.

6 Conclusion

This paper presented a `runc` fork that turns workload-specific profile generation into a runtime-side workflow. A single scanning flag records syscalls, capability checks, and AppArmor-relevant behaviour, then emits artefacts that can be used for later enforcement. The implementation keeps the scan explicit, keeps relaxation in memory, avoids generated executable hooks, and narrows capabilities according to observed behaviour.

The main result is architectural rather than a claim of completed benchmarking: profile generation can live inside the OCI runtime without requiring a Kubernetes recorder, a separate daemon, or manual policy authoring. The expected trade-off is a practical middle ground between default `runc` and heavier sandboxed runtimes. Future work should complete the benchmark matrix, add static augmentation for missed paths, emit Security Profiles Operator-compatible resources, and compare generated profiles head-to-head against cluster-side recorders.

References

1. Canella, C., et al.: Automating seccomp filter generation for linux applications. In: Proceedings of ACM CCS 2021. pp. 2137–2152. ACM (2021)
2. Cilium / Isovalent: Tetragon: ebpf-based security observability and runtime enforcement. `https://github.com/cilium/tetragon` (2025), accessed 2026-04-19
3. containers organisation: oci-seccomp-bpf-hook: Oci prestart hook that traces syscalls and emits a seccomp profile. `https://github.com/containers/oci-seccomp-bpf-hook` (2024), accessed 2026-04-19
4. Flauzac, O., Gonzalez, C., Nolot, F.: A review of native container security for running applications. International Journal of Computer Science and Engineering **22**(3), 187–204 (2020)
5. Inspektor Gadget contributors: Inspektor gadget: collection of ebpf-based tools for inspecting and debugging kubernetes workloads. `https://github.com/inspektor-gadget/inspektor-gadget` (2025), accessed 2026-04-19
6. IO Visor / iovisor: Bcc: Tools for bpf-based linux io analysis, networking, monitoring, and more (`capable`, `capable-bpfcc`). `https://github.com/iovisor/bcc` (2025), accessed 2026-04-19
7. Kubernetes SIG Auth, kubernetes-sigs: Security profiles operator. `https://github.com/kubernetes-sigs/security-profiles-operator` (2025), accessed 2026-04-19
8. Lopes, N., Martins, R., Correia, M.E., Serrano, S., Nunes, F.: Container hardening through automated seccomp profiling. In: Proceedings of the 6th International Workshop on Container Technologies and Container Clouds. pp. 31–36. ACM (2020). https://doi.org/10.1145/3429885.3429966

9. Mattetti, M., Allouche, Y.: Automatic security hardening and protection of linux containers. Security and Privacy in Communication Systems (2020)
10. Viktorsson, W., Klein, C., Tordsson, J.: Security-performance trade-offs of kubernetes container runtimes. In: 2020 IEEE MASCOTS. pp. 1–4. IEEE (2020)
11. Volpert, S., Winkelhofer, S., Wesner, S., Domaschka, J.: An empirical analysis of common oci runtimes' performance isolation capabilities. In: Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering. pp. 60–70. ACM (2024). https://doi.org/10.1145/3629526.3645044
12. Wang, K., et al.: Characterizing and optimizing kernel resource isolation for containerized applications. Future Generation Computer Systems **141**, 234–247 (2023)
13. Wang, X., Johannesson, D.: Performance and isolation analysis of runc, gvisor and kata containers runtimes. Cluster Computing **25**, 2363–2380 (2022). https://doi.org/10.1007/s10586-021-03517-8